

SRV6-CORE LAB · GUIDE

SRv6, from first principles to a working network

What Segment Routing over IPv6 is, how it carries VPNs across a provider core, and a guided tour of a lab where every piece of it runs on real routers.

PART 1

SRv6 in plain terms

The idea, the SID, the headers, uSID, the control plane, L3VPN — no prior SRv6 knowledge assumed.

PART 2

A tour of the lab

The topology and every tool around it: the portal, the looking glass, Nautobot, monitoring, tests.

PART 3

Walk a packet through it

Hands-on: from a ping on a tenant host, through every table and header, to steering it by hand.

Lab version 1.6.0 · 29 September 2026 · outputs captured from the running lab

Contents

PART 1 — SRV6 IN PLAIN TERMS

1. The problem SRv6 solves	3
2. Segment routing: put the route in the packet	3
3. The SID: an address that is an instruction	4
4. How the segments travel: the headers	5
5. Who tells whom: the control plane	5
6. uSID: many segments in one address	5
7. Putting it together: an SRv6 L3VPN	6
8. SRv6 compared with MPLS	7
9. Glossary	7

PART 2 — A TOUR OF THE LAB

10. What is in the box	8
11. Stop 1 — the provisioning portal	9
12. Stop 2 — the BGP looking glass	15
13. Stop 3 — Nautobot, the source of truth	18
14. Stop 4 — monitoring and alerting	20
15. Stop 5 — tests and CI	20

PART 3 — WALK A PACKET THROUGH IT

16. Before you start	21
17. Step 1 — the customer's view	21
18. Step 2 — the underlay: IS-IS and the locators	22
19. Step 3 — the instructions pe1 will carry out	22
20. Step 4 — BGP: which SID leads to which customer	23
21. Step 5 — the VRF: where encapsulation is decided	23
22. Step 6 — on the wire in the core	24
23. Step 7 — delivery at pe3	25
24. Step 8 — choose the path yourself	25
25. Step 9 — break something	26
26. Where to go next	26

How to read this guide. Part 1 stands on its own: read it first if SRv6 is new to you. Part 2 is a tour you can follow with the lab open in a browser. Part 3 is a walkthrough to do at a terminal; every output in it was captured from the running lab when this guide was built, so what you see on your screen should look the same, apart from times, counters and the function part of a SID (which the routers choose for themselves). The deeper companion, *SRv6 L3VPN, shown on real boxes* (docs/srv6-walkthrough.pdf), goes further into every table.

SRv6 in plain terms

1. The problem SRv6 solves

An ordinary IP router makes one decision per packet: *where is the destination, and which neighbour is closest to it?* Every router on the way makes that same decision again, independently, from its own routing table. That is simple and robust, and it is also all it can do. A service provider needs more:

- **Separation.** Many customers share one core, often with the same private addresses (two customers can both use `10.0.0.0/8`). Their traffic must never mix.
- **Choice of path.** Sometimes the shortest path is not the one you want — it is congested, too expensive, or you need two flows kept on separate links.
- **Fast recovery.** When a link fails, traffic should move in milliseconds, not in the seconds a routing protocol takes to reconverge.

For twenty years the answer was **MPLS**: put a small *label* in front of the packet and let every router forward on the label instead of the address. Labels mark which customer a packet belongs to (the VPN label) and which path it takes (the transport label). It works well, but it is a second forwarding plane next to IP, with its own signalling protocols (LDP, RSVP-TE) that every router must run and keep in step.

SRv6 does the same jobs with nothing but IPv6. The instructions that MPLS encoded as labels become IPv6 addresses. A router that only understands IPv6 can carry an SRv6 packet without knowing anything about it — and the routers that *do* need to act on it find the instruction in the destination address, where they look anyway.

2. Segment routing: put the route in the packet

Segment routing turns routing around. Instead of every router deciding the path on its own, the router where a packet **enters** the network decides, and writes the plan into the packet as an ordered list of instructions — **segments**. The routers along the way just execute the next instruction. They keep no per-flow or per-path state: all the state travels with the packet.

Hop-by-hop routing

every router decides again, from its own table



Packet: "to B" — and nothing else.
The path is whatever each table says.

Segment routing

the entry router writes the plan; the others execute it



Packet carries: [go via R2 , deliver at B]
each router pops the next instruction

Figure 1 — the same network, two ways of deciding the path. In segment routing the path is a property of the packet, not of the routers.

Think of it as a travel itinerary handed to a courier at the depot: *"to the Leeds hub, then the Manchester hub, then deliver to 12 High Street, office 4."* The hubs do not need to know about the parcel in advance; they read the next line of the itinerary.

Segment routing exists in two flavours: **SR-MPLS**, where each segment is an MPLS label, and **SRv6**, where each segment is an IPv6 address. This lab runs SRv6.

3. The SID: an address that is an instruction

In SRv6 each segment is called a **SID** (Segment Identifier), and a SID is a 128-bit IPv6 address. What makes an address a SID is only that some router has been told: *"when a packet arrives addressed to this, do the following."* That "following" is the SID's **behaviour**.

To keep this orderly, every SRv6 router owns a **locator** — a block of addresses announced into the routing protocol like any prefix, so the whole network knows how to reach it. The router then carves its SIDs out of its own locator. A SID therefore has two halves: the **locator** says *which router*, the **function** says *what to do there*.

A SID in this lab (format usid-f3216): 128 bits



Figure 2 — the SID fd00:c:3:e001:: read as an instruction: "go to pe3, and there apply function e001" (in this lab: hand the packet to one tenant's routing table).

Because the locator is an ordinary routed prefix, **a router that is not the SID's owner simply forwards the packet towards the locator**, exactly as it would forward any IPv6 packet. Only the owner looks at the function. This is the property everything else rests on.

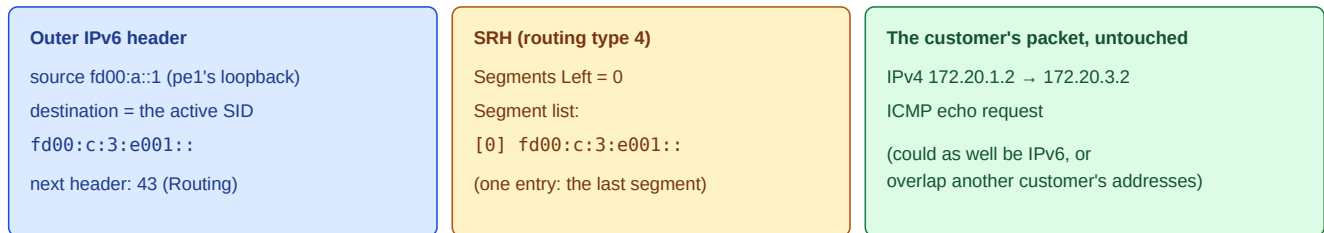
The behaviours you meet in this lab

Behaviour	Name in the lab	What the owner does with the packet	Used for
End	uN	"I was a waypoint": move on to the next segment and forward	a node on a chosen path
End.X	uA	move on, and send out of <i>this particular</i> link	a link on a chosen path; fast reroute
End.DT4 / DT6 / DT46	uDT46	remove the SRv6 wrapping and look the inner packet up in a given VRF	the end of a VPN: deliver to the customer
H.Encaps	(the PE's encapsulation)	wrap a customer packet in a new IPv6 header with a segment list	the start of a VPN or a steered path

The names come from RFC 8986 (*SRv6 Network Programming*). The "u" names are the micro-SID (uSID) versions of the same behaviours (section 6).

4. How the segments travel: the headers

When a customer packet enters the core, the entry router (here a PE) does not touch it. It puts it, whole, inside a new IPv6 packet — **encapsulation**. The outer header's *destination address* is the first SID. If more than one segment is needed, the rest go into an IPv6 extension header built for the purpose, the **Segment Routing Header (SRH, RFC 8754)**, with a counter, *Segments Left*, that says which segment is next.



What a transit router reads: only the outer destination — an IPv6 address inside pe3's locator. It routes it like any other.

What pe3 reads: the destination is its own SID with behaviour End.DT46 → strip the outer header and the SRH, deliver the inner packet.

Figure 3 — an SRv6 VPN packet as it crosses this lab's core (you will see exactly this in the p2 capture in Part 3).

Two consequences are worth holding on to:

1. **The core does not need to understand the customer.** The inner packet can be IPv4 or IPv6, and two customers can use the same addresses: they are inside different envelopes, addressed to different SIDs.
2. **The overhead is one IPv6 header (40 bytes) plus the SRH (8 bytes + 16 per segment).** That is why the core links in this lab run an MTU of 9000: a full 1500-byte customer packet must still fit after wrapping.

5. Who tells whom: the control plane

Routers still need to *learn* the SIDs. SRv6 reuses the protocols a provider runs anyway, with small extensions — no LDP, no RSVP.

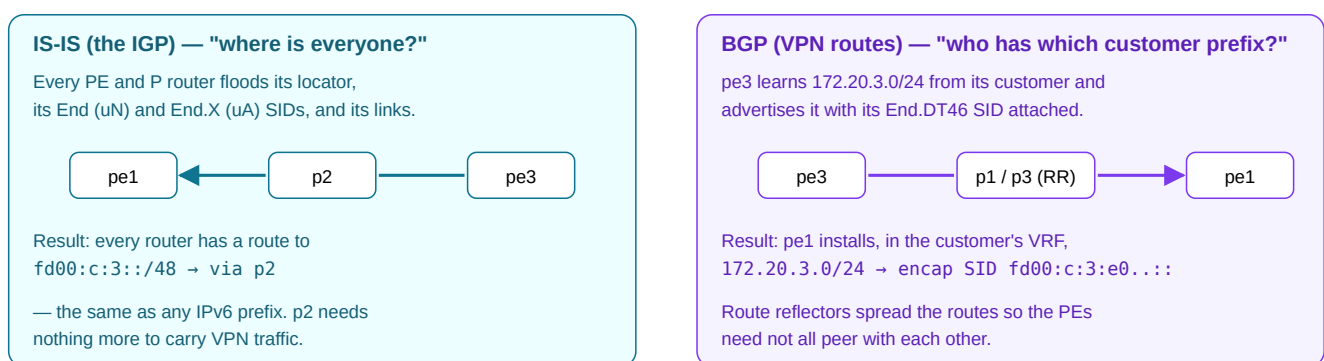


Figure 4 — two protocols, two questions. IS-IS makes the SIDs reachable; BGP says which SID leads to which customer prefix.

6. uSID: many segments in one address

A 128-bit SID per segment is generous. A three-hop path would need three full addresses in the SRH — 48 bytes of segments per packet. **Micro-SIDs (uSID)**, standardised as compressed SIDs in RFC 9800) pack several instructions into one address. In this lab each router's identity is only 16 bits (the *node* field), so after the 32-bit block there is room for up to **six** 16-bit micro-SIDs in one 128-bit address.

Each router that owns the next micro-SID does one simple thing: **shift the rest of the address left by 16 bits** and forward it again. The address itself becomes the itinerary.

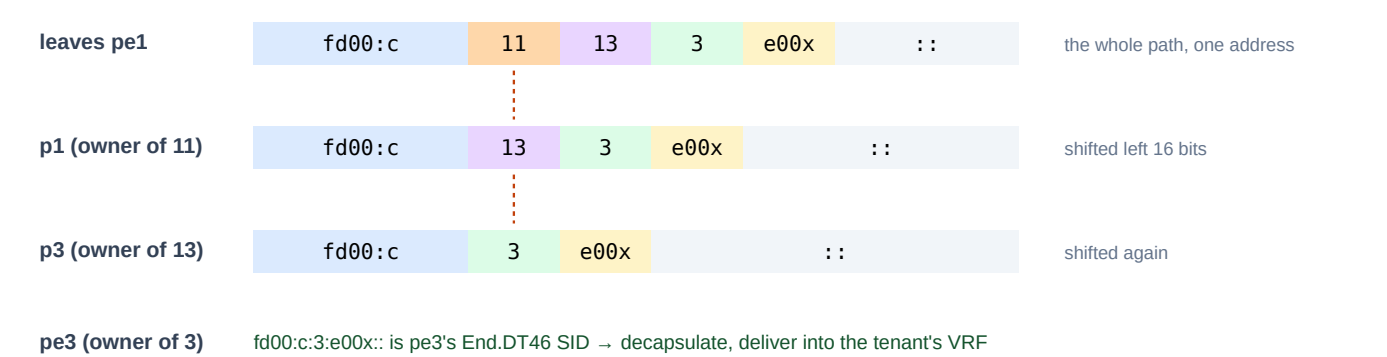


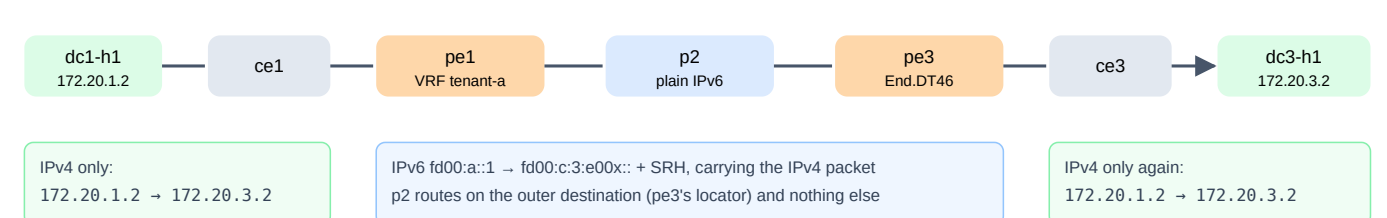
Figure 5 — the steered path $pe1 \rightarrow p1 \rightarrow p3 \rightarrow pe3$ as a single uSID "carrier" (Part 3, step 8, shows these three addresses on the wire).

The SRH shrinks from three segments to one, and only the destination address changes hop by hop — which is what every router's hardware already rewrites cheaply. In this lab the kernel calls this the *NEXT-C-SID* flavour of End; the lab's steering tool can also install the classic uncompressed three-segment list, so you can compare the two.

7. Putting it together: an SRv6 L3VPN

A **Layer-3 VPN** gives each customer (a *tenant*) its own private routing domain across the shared core. The building blocks:

Piece	What it is	In this lab
VRF	a separate routing table on the PE for one customer	tenant-a (table 100), tenant-b (table 200) on every PE and CE
Attachment circuit	the link from the customer's router (CE) into that VRF	172.16.n.0/30 (tenant-a), 172.18.n.0/30 (tenant-b)
RD (route distinguisher)	a prefix that keeps overlapping customer routes apart inside BGP	65000:10n, 65000:20n
RT (route target)	a tag saying which VRFs import a route — the actual membership	65000:100, 65000:200
Service SID	the End.DT46 SID that leads into that VRF on that PE	one per tenant per PE, allocated by BGP



A tenant-b packet takes the same core path in a different envelope: pe3's *other* End.DT46 SID, which leads into VRF tenant-b. No tenant-a route exists in any tenant-b table, so the two can never reach each other — not even at the same site.

Figure 6 — one ping across the lab, and what the packet looks like on each stretch.

8. SRv6 compared with MPLS

	MPLS L3VPN	SRv6 L3VPN (this lab)
Forwarding in the core	on labels; every core router must run MPLS	on IPv6 addresses; the core only needs IPv6 routing
Transport signalling	LDP or RSVP-TE on every router	none — IS-IS announces locators
Customer identity	a VPN label from BGP	a service SID from BGP (End.DT4/DT6/DT46)
Choosing a path	RSVP-TE tunnels (state on every hop) or SR-MPLS	a segment list in the packet; no state in the core
Header cost	4 bytes per label	40-byte IPv6 header + SRH (uSID keeps it small)
Troubleshooting	label tables, <code>show mpls</code>	ordinary IPv6 tools: ping, traceroute, tcpdump see the SIDs

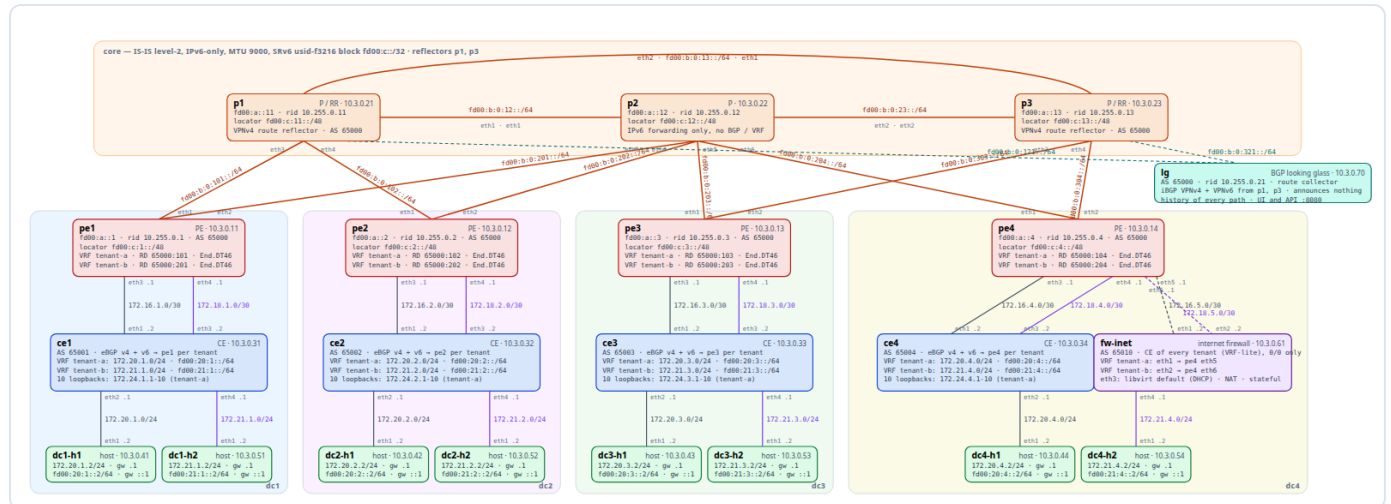
9. Glossary

Term	Meaning
SID	Segment Identifier — an IPv6 address that names an instruction on one router
Locator	the part of a SID that says which router; announced by IS-IS (<code>fd00:c:3::/48</code> = pe3)
Function	the part of a SID that says what to do there (<code>e001</code>)
Behaviour	the standard action behind a function: End, End.X, End.DT46 ... (RFC 8986)
SRH	Segment Routing Header — the IPv6 extension header that holds the segment list (RFC 8754)
uSID / C-SID	micro-SID: 16-bit instructions packed several to an address, consumed by shifting (RFC 9800)
uN / uA / uDT46	the micro-SID forms of End, End.X and End.DT46
PE / P / CE	Provider Edge (has the customer VRFs), Provider core router (only IPv6), Customer Edge (the customer's router)
VRF	a separate routing table for one customer on a PE or CE
RD / RT	Route Distinguisher (keeps routes apart in BGP) / Route Target (decides which VRFs import a route)
RR	Route Reflector — a BGP router that relays VPN routes so PEs need not all peer with each other
Encapsulation	wrapping the customer's packet in a new IPv6 header (H.Encaps)

A tour of the lab

10. What is in the box

The lab is a small service-provider network, simulated on one Linux host with KVM. Every router is a real **VyOS** router (FRR inside) running its production software; every customer host is a real **Alpine Linux** VM.



The *srv6-core* topology, generated from *lab.conf*

Role	Nodes	What they do
P routers	p1, p2, p3	the core triangle: IS-IS, SRv6, plain IPv6 forwarding. p1 and p3 are also BGP route reflectors; p2 runs no BGP at all
PE routers	pe1 – pe4	one per data centre: the tenant VRFs, the eBGP sessions to the CEs, SRv6 encapsulation and decapsulation
CE routers	ce1 – ce4	the customer side of each site: one VRF per tenant, a LAN per tenant, ten extra loopbacks for tenant-a
Hosts	dcn-h1 (tenant-a), dcn-h2 (tenant-b)	Alpine Linux with ping, traceroute, iperf3, tcpdump — the "customers"
fw-inet	one firewall on pe4	a CE of every tenant that gives each site a NAT'd way out to the internet
lg	the looking glass	a passive BGP route collector with history, packet capture and path tracing
NMS	Prometheus, VictoriaMetrics, Grafana, Gitea	monitoring, alerting, the CI runner's Git mirror

pe1 and pe2 hang off p1 and p2; pe3 and pe4 off p2 and p3. So every west ↔ east path crosses **p2** by default — a fact the tests and the steering examples use.

Addressing at a glance

What	Range	Example
Loopbacks (core)	fd00:a::/48	pe1 fd00:a::1, p2 fd00:a::12
Core links	fd00:b:0:<ab>::/64	p2-pe1 fd00:b:0:201::/64
SRv6 locators	fd00:c:<node>::/48 from block fd00:c::/32	pe3 fd00:c:3::/48, p1 fd00:c:11::/48
tenant-a	circuits 172.16.n.0/30, LANs 172.20.n.0/24	dc3-h1 = 172.20.3.2
tenant-b	circuits 172.18.n.0/30, LANs 172.21.n.0/24	dc3-h2 = 172.21.3.2
IPv6 twins	172.X.Y.0 ↔ fd00:X:Y::/64	LAN 172.20.1.0/24 ↔ fd00:20:1::/64
Management (OOB)	10.3.0.0/24	pe1 10.3.0.11, dc1-h1 10.3.0.41

Driving it

```
./lab.sh up          # start every VM (about six minutes from cold)
./lab.sh status      # every node and link with its addresses
./lab.sh ssh pe1     # a router (vyos / vyos); hosts are lab / lab
./lab.sh verify      # adjacencies, SIDs, VPN routes, the host ping matrix
./lab.sh test        # the Robot Framework suites
./lab.sh down        # shut everything down; configurations are saved
```

11. Stop 1 — the provisioning portal

The portal (<http://<lab-host>:8091>) is where tenants are created, changed and watched. Everything it does goes through the same steps an engineer would take — edit `lab.conf`, render configurations, push them over SSH, update Nautobot, verify, test — as a **run** you can follow line by line.

Tenants. One card per tenant with every site: the PE port, route distinguisher, attachment circuit, CE ports, LAN and host, joined with live state read from the routers — the eBGP session to the CE, how many routes the VRF holds, how many of them go through SRv6, and the tenant's End.DT46 SID on that PE.

SRv6 Tenant Provisioning Portal

VyOS SRv6 core · IS-IS + BGP L3VPN (End.DT4) · Nautobot source of truth · Robot Framework validation

Tenants

Steering

SLA

Capacity

Backups

Runs

API

Looking glass

Nautobot: [open](#) · [dark](#) · [Default](#)

2

Tenants

8

Sites

8 / 8

Hosts reachable

4

PEs

p1, p3

Route reflectors

0

Steering policies

Tenants

one VRF per tenant on every PE and CE; tenants never meet

Refresh live state

Add tenant

tenant-a

up

table 100 · RT 65000:100 · [VRF](#) · [tenant](#) · [prefixes](#)

Throughput

Add site

Remove

Site	PE	RD	Attachment circuit	CE	LAN	Host	eBGP CE -- PE	VRF routes (SRv6)	End.DT4 SID
dc1	pe1 eth3	65000:101	172.16.1.0/30 fd00:16:1::/64	ce1 eth1 / eth2	172.20.1.0/24 fd00:20:1::/64	dc1-h1 up 172.20.1.2 · fd00:20:1::2 · oob 10.3.0.41	Established v6 up 11 pfx	49 (25)	fd00:c:1:e002::
dc2	pe2 eth3	65000:102	172.16.2.0/30 fd00:16:2::/64	ce2 eth1 / eth2	172.20.2.0/24 fd00:20:2::/64	dc2-h1 up 172.20.2.2 · fd00:20:2::2 · oob 10.3.0.42	Established v6 up 11 pfx	49 (25)	fd00:c:2:e001::
dc3	pe3 eth3	65000:103	172.16.3.0/30 fd00:16:3::/64	ce3 eth1 / eth2	172.20.3.0/24 fd00:20:3::/64	dc3-h1 up 172.20.3.2 · fd00:20:3::2 · oob 10.3.0.43	Established v6 up 11 pfx	49 (24)	fd00:c:3:e001::
dc4	pe4 eth3	65000:104	172.16.4.0/30 fd00:16:4::/64	ce4 eth1 / eth2	172.20.4.0/24 fd00:20:4::/64	dc4-h1 up 172.20.4.2 · fd00:20:4::2 · oob 10.3.0.44	Established v6 up 11 pfx	49 (24)	fd00:c:4:e001::

tenant-b

up

table 200 · RT 65000:200 · [VRF](#) · [tenant](#) · [prefixes](#)

Throughput

Add site

Remove

Site	PE	RD	Attachment circuit	CE	LAN	Host	eBGP CE -- PE	VRF routes (SRv6)	End.DT4 SID
dc1	pe1 eth4	65000:201	172.18.1.0/30 fd00:18:1::/64	ce1 eth3 / eth4	172.21.1.0/24 fd00:21:1::/64	dc1-h2 up 172.21.1.2 · fd00:21:1::2 · oob 10.3.0.51	Established v6 up 1 pfx	9 (5)	fd00:c:1:e003::
dc2	pe2	65000:202	172.18.2.0/30	ce2 eth3 /	172.21.2.0/24	dc2-h2 up	Established v6 up 1	9 (5)	fd00:c:2:e000::

The Tenants view: every site with its live state

Topology. The same network drawn live, hosts coloured by whether they answer. **Add tenant** opens a three-step wizard that suggests everything (letter, table, route target, ports, addresses, hosts), re-validates it against the running lab, and deploys it as a run.

SRv6 Lab Guide · srv6-core 1.6.0

10

Add a tenant

dc1 PE pe1, CE ce1

PE port:
 CE port → PE:
 CE LAN port:
 Attachment circuit /30:

LAN /24:

Host name:
 Host OOB address:
 Console port:
 Node index:

dc2 PE pe2, CE ce2

PE port:
 CE port → PE:
 CE LAN port:
 Attachment circuit /30:

LAN /24:

Host name:
 Host OOB address:
 Console port:
 Node index:

dc3 PE pe3, CE ce3

PE port:
 CE port → PE:
 CE LAN port:
 Attachment circuit /30:

LAN /24:

Host name:
 Host OOB address:
 Console port:
 Node index:

Adding a tenant: the per-site allocation step

Steering — on the map. Pin a tenant prefix on a PE to a path through chosen P routers. **Preview** draws the path before it is applied: the dashed grey line is the IGP shortest path, the coloured one the path the policy would take. After **Apply**, **Measure delay** pings both from the PE inside the tenant's VRF.

On the map each policy's steered path beside the IGP shortest path it replaces, with the delay of each, measured from the PE in the tenant's VRF

Show

--- IGP shortest path — steered — preview (not applied)

Path	Hops	Delay
--- pe1 → p2 → pe3	2	—
— pe1 → p1 → p3 → pe3	3	apply it to measure

Preview — not applied. 1 hop(s) longer than the IGP.

Paths come from the model (every IS-IS metric is the default, so the IGP takes the fewest hops; equal-cost paths are all dashed). Delay: ten echo requests 0.2 s apart from the PE, inside the tenant's VRF — to the host on the steered prefix (the policy's path) and to the same site's CE on its attachment circuit (a prefix no policy steers: the IGP path to the same PE).

The segment list is the End SID of every listed P router followed by the End.DT4 SID of the PE that owns the prefix (read live). Return traffic keeps the IGP shortest path. Suite 07 shows the SRH on the wire.

A steering preview: pe1 → p1 → p3 → pe3 beside the IGP path via p2

SLA. Every minute each tenant host pings every other site of its tenant, and each pair is graded against delay and loss targets. The matrix shows the last probe; a pair's history is one click away. A pair that changes state is annotated in Grafana, and alert rules fire on the same numbers.

SRv6 Tenant Provisioning Portal

VyOS SRv6 core · IS-IS + BGP L3VPN (End,DT4) · Nautobot source of truth · Robot Framework validation

TenantsSteeringSLACapacityBackupsRunsAPILooking glass

Nautobot: [open](#) · [dark](#) · [Default](#)

24

site pairs probed

24

meet their SLA

0

warning

0

critical

2.75 ms

worst RTT

8:18:18 AM

last probe

Tenant SLA

every pair of a tenant's sites, probed host to host every minute

Probe now

tenant-a

all pairs within SLA

12 pairs · worst 2.643 ms · worst loss 0 %

from \ to	dc1-h1 dc1	dc2-h1 dc2	dc3-h1 dc3	dc4-h1 dc4
dc1-h1 dc1	.	2.64 ms 0 % loss	2.52 ms 0 % loss	2.45 ms 0 % loss
dc2-h1 dc2	2.21 ms 0 % loss	.	2.44 ms 0 % loss	2.60 ms 0 % loss
dc3-h1 dc3	2.38 ms 0 % loss	2.09 ms 0 % loss	.	2.62 ms 0 % loss
dc4-h1 dc4	2.40 ms 0 % loss	2.49 ms 0 % loss	2.57 ms 0 % loss	.

tenant-b

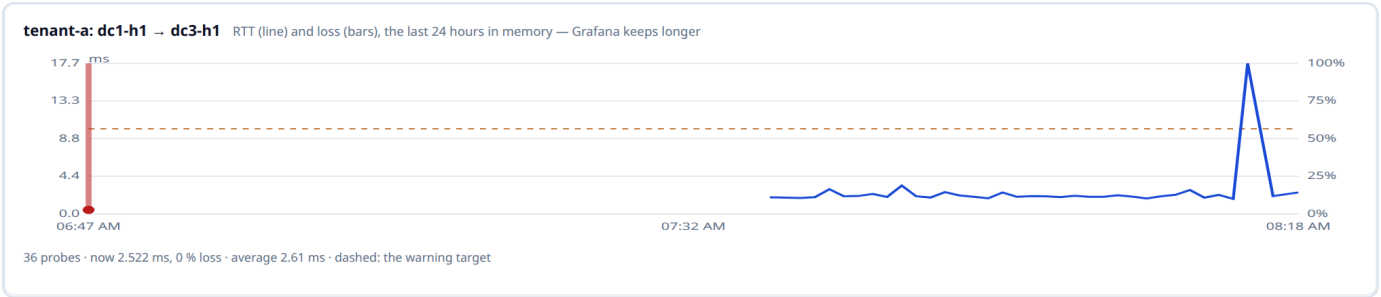
all pairs within SLA

12 pairs · worst 2.755 ms · worst loss 0 %

from \ to	dc1-h2 dc1	dc2-h2 dc2	dc3-h2 dc3	dc4-h2 dc4
dc1-h2 dc1	.	2.44 ms 0 % loss	2.75 ms 0 % loss	2.65 ms 0 % loss
dc2-h2 dc2	2.60 ms 0 % loss	.	2.25 ms 0 % loss	2.04 ms 0 % loss
dc3-h2 dc3	2.54 ms 0 % loss	2.70 ms 0 % loss	.	2.33 ms 0 % loss
dc4-h2 dc4	2.29 ms 0 % loss	2.71 ms 0 % loss	2.02 ms 0 % loss	.

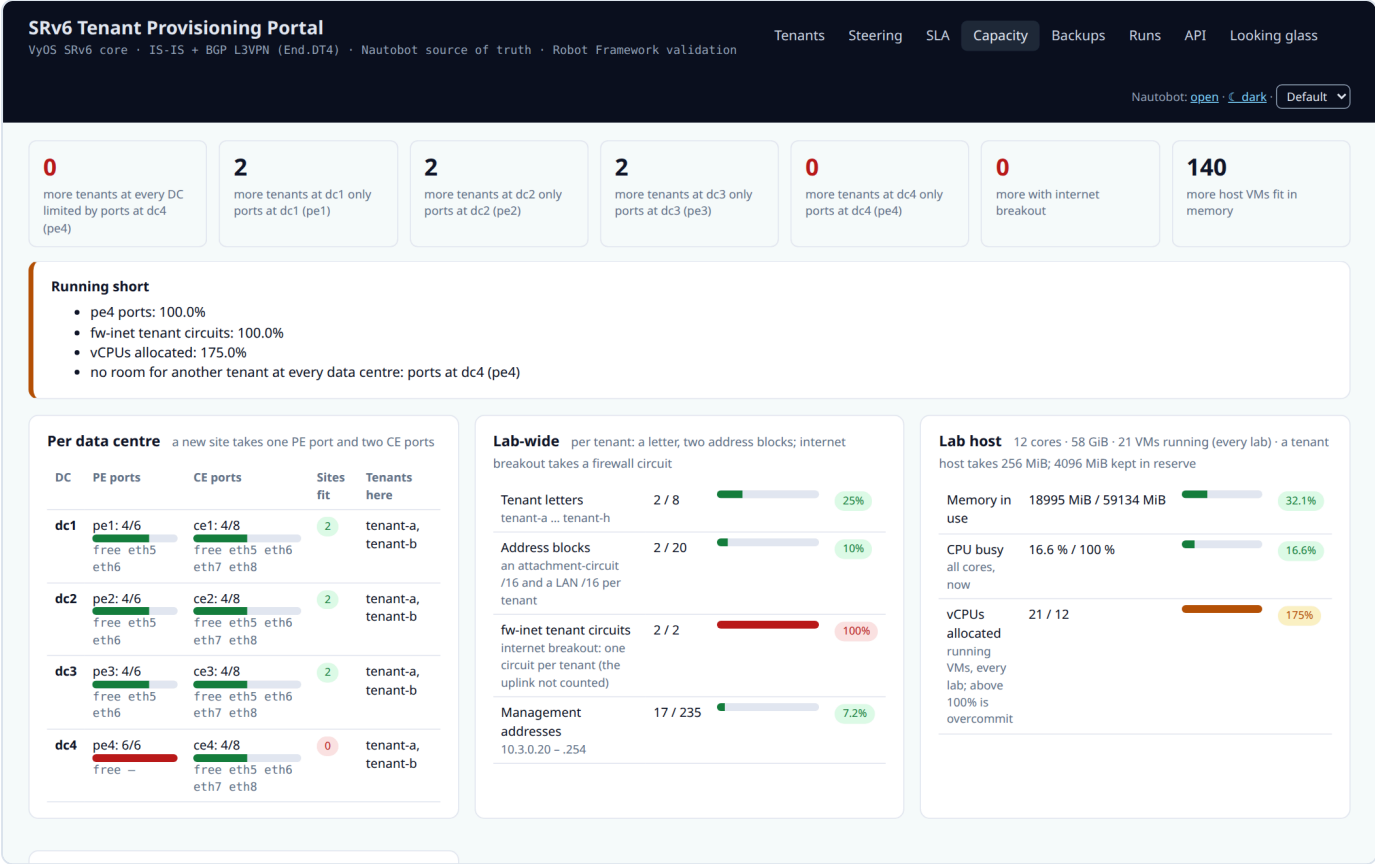
Targets: RTT below 10 ms and loss below 1 % (warning above); critical at 25 ms or 5 % loss, or no answer. 10 echo requests per pair every 60 s. A pair that changes state writes a Grafana annotation; the alert rules TenantSLaBreach / TenantSLaDown fire from the same numbers.

The SLA matrix: every pair of every tenant's sites



One pair's delay and loss over time

Capacity. How many more tenants fit — at every data centre and at each one — and which limit runs out first: PE or CE ports, tenant letters, address blocks, firewall circuits, host memory.



Capacity: room to grow, and the limit that binds first

Backups. The whole lab state in one checksummed file (`lab.conf` , the steering policies, every router's configuration). A restore first shows its plan — tenants to add or remove, steering to put back — and then runs like any other change.

SRv6 Tenant Provisioning Portal

VyOS SRv6 core · IS-IS + BGP L3VPN (End.DT4) · Nautobot source of truth · Robot Framework validation

TenantsSteeringSLACapacityBackupsRunsAPILooking glass

Nautobot: [open](#) · [dark](#) · [Default](#)

Backups

the whole lab in one file: lab.conf, the steering policies, every router's running and rendered configuration

Choose FileNo file chosenBack up now

Taken	Version	Tenants	Steering	Size		
9/28/2026, 8:06:08 PM srv6-core-20260928-200608.tar.gz	1.3.0	tenant-a, tenant-b	1	21 kB	Download	Plan restore
9/28/2026, 7:46:27 PM srv6-core-20260928-194627.tar.gz	1.2.0	tenant-a, tenant-b	1	21 kB	Download	Plan restore
9/28/2026, 7:37:19 PM srv6-core-20260928-193719.tar.gz	1.2.0	tenant-a, tenant-b	0	21 kB	Download	Plan restore
9/28/2026, 7:01:33 PM srv6-core-20260928-190133.tar.gz	1.2.0	tenant-a, tenant-b	0	21 kB	Download	Plan restore

Restore plan

srv6-core-20260928-200608.tar.gz · taken 9/28/2026, 8:06:08 PM · v1.3.0

Add tenants

nothing

Add sites

nothing

Remove tenants

nothing

Steering

- + pe1 tenant-b 172.21.3.0/24 via p1 p3

☒ run the Robot suites at the end

Restore

A restore changes tenants and steering only; tenants the backup has and the lab does not are added with the backup's own ports, addresses and hosts; tenants the lab has and the backup does not are removed; the steering policies become the backup's. It refuses when the core differs, or when a tenant would have to lose a single site. Nautobot is re-seeded from lab.conf; the looking glass's history is not in a backup.

A backup and its restore plan

Runs. Every change as steps with a streamed log and, at the end, the Robot Framework report. A failed run can be resumed from the step that failed.

SRv6 Tenant Provisioning Portal

VyOS SRv6 core · IS-IS + BGP L3VPN (End.DT4) · Nautobot source of truth · Robot Framework validation

TenantsSteeringSLACapacityBackupsRunsAPILooking glass

Nautobot: [open](#) · [dark](#) · [Default](#)

Runs

one at a time

Run the test suites

Started	Mode	Tenant	Status	Tests
9/28/2026, 8:07:30 PM	restore	restore srv6-core-20260928-200608.tar.gz	success	
9/28/2026, 7:47:48 PM	restore	restore srv6-core-20260928-194627.tar.gz	success	
9/28/2026, 7:26:11 PM	restore	restore srv6-core-20260928-190133.tar.gz	success	
9/28/2026, 7:19:15 PM	restore	restore srv6-core-20260928-190133.tar.gz	failed	
9/16/2026, 11:36:56 PM	test		success	3/3
9/16/2026, 8:07:19 PM	remove	tenant-c	success	16/16
9/16/2026, 7:56:04 PM	remove	tenant-c	failed	
9/16/2026, 7:55:16 PM	remove	tenant-c	failed	
9/16/2026, 7:25:37 PM	tenant	tenant-c	success	52/52
9/16/2026, 7:09:14 PM	tenant	tenant-c	failed	

tenant tenant-c success 2026-09-16_19-25-37-0348

success

Validate the allocation

(from run 2026-09-16_19-09-14-f4cf) tenant-c: 4 site(s) — dc1 (dc1-h3, 172.22.1.0/24), dc2 (dc2-h3, 172.22.2.0/24), dc3 (dc3-h3, 172.22.3.0/24), dc4 (dc4-h3, 172.22.4.0/24)

success

Register in lab.conf, render the day-0 configs

(from run 2026-09-16_19-09-14-f4cf) hosts dc1-h3, dc2-h3, dc3-h3, dc4-h3; CEs ce1, ce2, ce3, ce4

success

Create and boot the host VMs

(from run 2026-09-16_19-09-14-f4cf) dc1-h3, dc2-h3, dc3-h3, dc4-h3

success

Re-wire the CE VMs (new attachment circuit and LAN ports)

(from run 2026-09-16_19-09-14-f4cf) ce1, ce2, ce3, ce4 re-wired and back

success

Push the configuration to the PEs and CEs (SSH)

(from run 2026-09-16_19-09-14-f4cf) ce1, ce2, ce3, ce4, pe1, pe2, pe3, pe4

success

Nautobot source of truth (seed)

(from run 2026-09-16_19-09-14-f4cf) seeded

success

Verify: tenant ping matrix, Nautobot rendering == lab.conf

tenant reachable, Nautobot == lab.conf

success

Robot Framework tests

52/52 passed

Tests: 52/52 passed · [report](#) · [log](#)

04 Vpn 5/5
05 End To End 4/4
09 Nautobot 7/7
01 Management 3/3
02 Underlay 3/3
03 Srv6 4/4
04 Vpn 5/5
05 End To End 4/4
06 Rr Redundancy 3/3
07 Steering 4/4
08 Failover 3/3

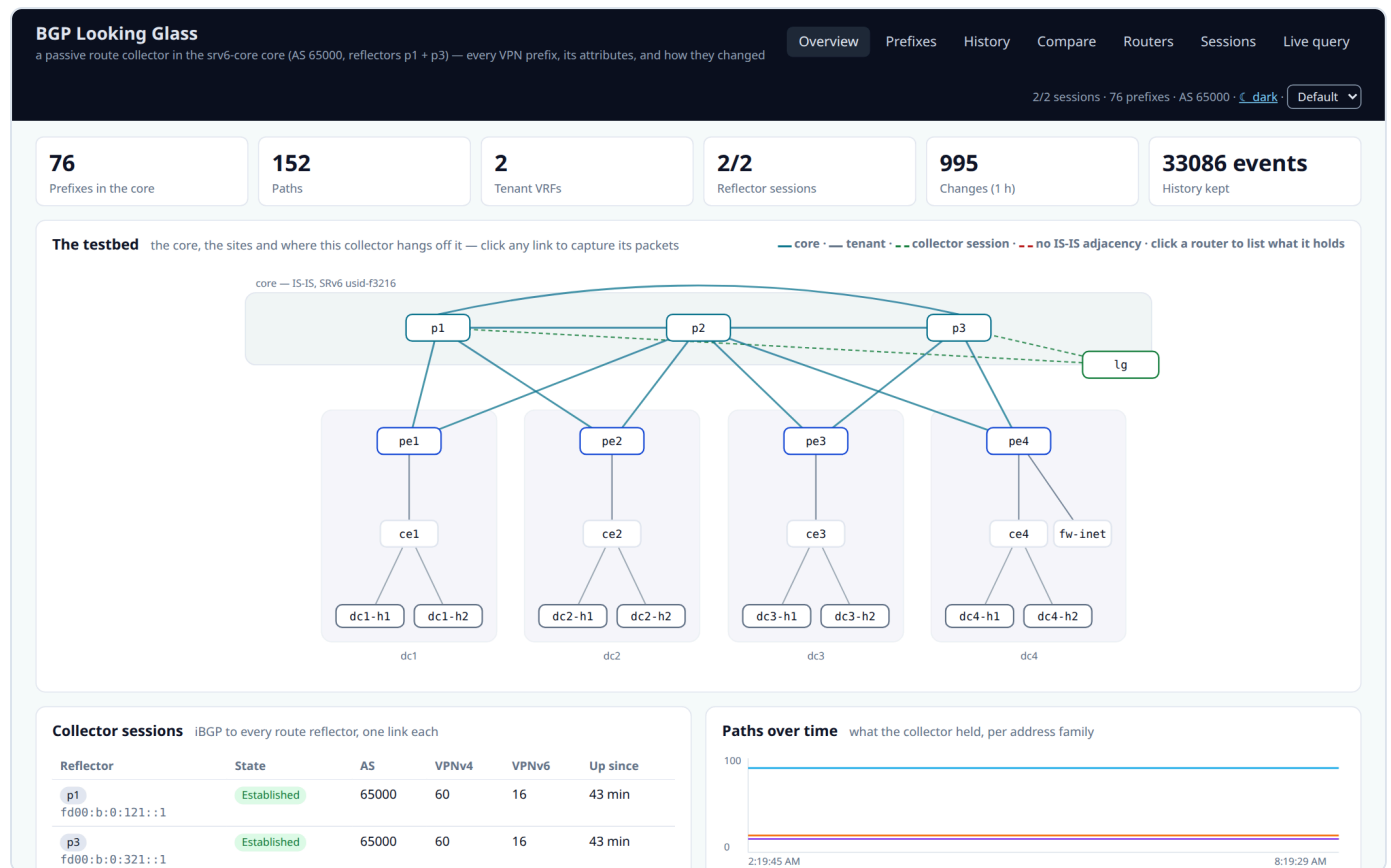
A run: steps, log and test results

Themes. The picker in the portal's header switches between the default layout and an Apple-style theme; ☾ / ✱ switches dark and light in either. The looking glass has the same picker.

12. Stop 2 — the BGP looking glass

The looking glass (`http://10.3.0.70:8080`) is a **passive route collector** inside the core. It peers with both route reflectors, receives every VPN route and announces nothing. It also reads each router's own tables over the router API. Everything it sees is kept as history, so you can ask what the network looked like at any moment.

Overview. The collector's sessions, the size of the VPN table, recent churn, and the topology — click any link to capture its packets.



The looking glass overview

Prefixes. Every VPN prefix with its route distinguisher, route target, next hop and the SRv6 SID it points to, filterable by VRF, RD, origin AS or free text.

BGP Looking Glass

a passive route collector in the srv6-core core (AS 65000, reflectors p1 + p3) — every VPN prefix, its attributes, and how they changed

Overview

Prefixed

History

Compare

Routers

Sessions

Live query

2/2 sessions · 76 prefixes · AS 65000 · [dark](#) · [Default](#)

Prefixed

3086 paths match

View

Address family

VRF

Route distinguisher

How it was read

Origin AS

Search (prefix, next hop, SID, RT, AS path)

every view

every family

every VRF

every RD

however it was

65001

172.20. / fd00:c1:: / RT:65000:100

Search

Reset

The collector's view is the core's VPN table: one row per path a reflector handed over, with the attributes the PE originated.

	Prefix	VRF	RD	Next hop	From	AS path	Origin	Local pref	MED	SID	Route targets	Age	View	How
path	0.0.0.0/0 best	default	-	192.168.122.1 fw-inet	local	i	-	-	-	-	-	4 d	fw-inet	router API
path	0.0.0.0/0 best	tenant-a	-	172.16.1.1 ce1	local	i	-	-	-	-	-	4 d	ce1	router API
path	0.0.0.0/0 best	tenant-a	-	172.16.1.1	172.16.1.1	65000 65010	65010	-	0	-	RT:65000:100	5 d	ce1	router SSH
path	0.0.0.0/0 best	tenant-a	-	172.16.2.1 ce2	local	i	-	-	-	-	-	4 d	ce2	router API
path	0.0.0.0/0 best	tenant-a	-	172.16.2.1	172.16.2.1	65000 65010	65010	-	0	-	RT:65000:100	5 d	ce2	router SSH
path	0.0.0.0/0 best	tenant-a	-	172.16.3.1 ce3	local	i	-	-	-	-	-	4 d	ce3	router API
path	0.0.0.0/0 best	tenant-a	-	172.16.3.1	172.16.3.1	65000 65010	65010	-	0	-	RT:65000:100	5 d	ce3	router SSH
path	0.0.0.0/0 best	tenant-a	-	172.16.4.1 ce4	local	i	-	-	-	-	-	4 d	ce4	router API
path	0.0.0.0/0 best	tenant-a	-	172.16.4.1	172.16.4.1	65000 65010	65010	-	-	-	-	5 d	ce4	router SSH
path	0.0.0.0/0 best	tenant-a	65000:104	fd00:a::4 pe4	p1	65010	65010	100	0	fd00:c4::	RT:65000:100	5 d	core	session

Load more

showing 200 of 3086

Every VPN prefix and its SRv6 attributes

Path. For one prefix, the path a packet really takes: from the ingress PE's VRF, through the P router the PE actually forwards through, to the egress PE and its SID — read from the routers, not guessed.

BGP Looking Glass

a passive route collector in the srv6-core core (AS 65000, reflectors p1 + p3) — every VPN prefix, its attributes, and how they changed

Overview

Prefixed

History

Compare

Routers

Sessions

Live query

2/2 sessions · 76 prefixes · AS 65000 · [dark](#) · [Default](#)

Prefixed

3086 paths match

View

Address family

VRF

Route distinguisher

How it was read

Origin AS

Search (prefix, next hop, SID, RT, AS path)

every view

every family

every VRF

every RD

however it was

65001

172.20. / fd00:c1:: / RT:65000:100

Search

Reset

The collector's view is the core's VPN table: one row per path a reflector handed over, with the attributes the PE originated.

The path to 172.20.3.0/24

router by router, from a chosen vantage point

from **pe1** [close](#)

172.20.3.0/24 in tenant-a · RD 65000:103 · RT:65000:100 · egress **pe3** · SID fd00:c3::3 · [IGP shortest path](#)

Capture along this path

who said so: the route and its attributes from [session](#), what pe1 actually forwards out of from [router API](#)

core — IS-IS, SRv6 usid-f3216

1. dc1-h1 host

sends to 172.20.3.0/24

172.20.1.2 on 172.20.1.0/24

2. ce1 ce eth1

the tenant-a CE at dc1

eBGP to pe1 over 172.16.1.0/30

3. pe1 pe eth3

ingress PE: imports the route into VRF

tenant-a

4. p2 p eth3

P router: forwards the encapsulated

packet

The path to 172.20.3.0/24, hop by hop

History and Compare. Every announce, change and withdraw, with a time slider that shows any table as it was. Compare lists every path that moved between two moments, down to a single next hop's SID.

BGP Looking Glass

a passive route collector in the srv6-core core (AS 65000, reflectors p1 + p3) — every VPN prefix, its attributes, and how they changed

Overview

Prefixes

History

Compare

Routers

Sessions

Live query

2/2 sessions · 76 prefixes · AS 65000 · [dark](#) · [Default](#)

History

the whole table's changes, newest first

Prefix (optional)

172.20.3.0/24

Kind

any

Since

last day

Show

When		Prefix	VRF	View	From	What changed
9/29/2026, 8:18:33 AM	change	fd00:21:1::/64	tenant-b	pe4	pe1	label: 917552 → 917520; transposed_sid: "fd00:c:1:e003:" → "fd00:c:1:e001:"
9/29/2026, 8:18:33 AM	change	fd00:21:1::/64	tenant-b	pe4	pe1	label: 917552 → 917520; transposed_sid: "fd00:c:1:e003:" → "fd00:c:1:e001:"
9/29/2026, 8:18:33 AM	change	fd00:18:1::/64	tenant-b	pe4	pe1	label: 917552 → 917520; transposed_sid: "fd00:c:1:e003:" → "fd00:c:1:e001:"
9/29/2026, 8:18:33 AM	change	fd00:18:1::/64	tenant-b	pe4	pe1	label: 917552 → 917520; transposed_sid: "fd00:c:1:e003:" → "fd00:c:1:e001:"
9/29/2026, 8:18:33 AM	change	fd00:21:1::/64	tenant-b	pe2	pe1	label: 917552 → 917520; transposed_sid: "fd00:c:1:e003:" → "fd00:c:1:e001:"
9/29/2026, 8:18:33 AM	change	fd00:18:1::/64	tenant-b	pe2	pe1	label: 917552 → 917520; transposed_sid: "fd00:c:1:e003:" → "fd00:c:1:e001:"
9/29/2026, 8:18:33 AM	change	fd00:18:1::/64	tenant-b	pe2	pe1	label: 917552 → 917520; transposed_sid: "fd00:c:1:e003:" → "fd00:c:1:e001:"
9/29/2026, 8:18:33 AM	change	172.21.1.0/24	tenant-b	pe2	pe1	label: 917552 → 917520; transposed_sid: "fd00:c:1:e003:" → "fd00:c:1:e001:"
9/29/2026, 8:18:33 AM	change	172.21.1.0/24	tenant-b	pe2	pe1	label: 917552 → 917520; transposed_sid: "fd00:c:1:e003:" → "fd00:c:1:e001:"
9/29/2026, 8:18:33 AM	change	172.20.3.0/24	tenant-b	pe2	pe1	label: 917552 → 917520; transposed_sid: "fd00:c:1:e003:" → "fd00:c:1:e001:"

History: what changed, and when

Packet capture. Click any link on the map to capture on it — live, streamed into the page — or capture along a prefix's whole path at once and watch each ping hop by hop: plain IPv4 on the access links, SRv6-encapsulated in the core.

BGP Looking Glass

a passive route collector in the cloud

Overview

Prefixes

History

Routers

Sessions

Live query

76 prefixes · AS 65000 · [c_dark](#)

Reset

Prefixes

3087 paths made

View

every view

The collector's view is the collector's view

The path to 172.20.3.0/24

172.20.3.0/24 in tenant-a

who said so: the route and its origin

Capture along the path to 172.20.3.0/24

dc1-h1 → ce1 → pe1 → p2 → pe3 → ce3 → dc3-h1 · tenant-a · SID fd00:c::3::

	Hop	Captured at	Keep
☒	1. dc1-h1 → ce1	ce1 eth2	pings
☒	2. ce1 → pe1	ce1 eth1	pings
☒	3. pe1 → p2	pe1 eth2	SRv6-encapsulated
☒	4. p2 → pe3	p2 eth5	SRv6-encapsulated
☒	5. pe3 → ce3	pe3 eth3	pings

Seconds (at most)

Packets per point

12

100

☒ Send 5 pings from ce1 (VRF tenant-a) to 172.20.3.2, the destination host

Start

Close

done · ping: 5 packets transmitted, 5 received, 0% packet loss, time 2005ms · rtt min/avg/max/mdev = 1.771/1.826/1.873/0.038 ms

1. ce1 eth2 · 0 · done · pcap

2. ce1 eth1 · 10 · done · pcap

3. pe1 eth2 · 10 · done · pcap

4. p2 eth5 · 10 · done · pcap

5. pe3 eth3 · 14 · done · pcap

6. ce3 eth2 · 10 · done · pcap

The journey of each ping — by hop, left to right; the times are each router's own clock

Ping	1. dc1-h1 → ce1 ce1 eth2	2. ce1 → pe1 ce1 eth1	3. pe1 → p2 pe1 eth2	4. p2 → pe3 p2 eth5	5. pe3 → ce3 pe3 eth3	6. ce3 → dc3-h1 ce3 eth2
reply 1	before the pings	✓ 29.829 ICMP	✓ 30.423 SRv6	✓ 29.387 SRv6	✓ 29.643 ICMP	✓ 30.061 ICMP
request 1	before the pings	✓ 29.827 ICMP	✓ 30.421 SRv6	✓ 29.386 SRv6	✓ 29.642 ICMP	✓ 30.060 ICMP
reply 2	before the pings	✓ 30.329 ICMP	✓ 30.923 SRv6	✓ 29.888 SRv6	✓ 30.143 ICMP	✓ 30.561 ICMP
request 2	before the pings	✓ 30.327 ICMP	✓ 30.921 SRv6	✓ 29.887 SRv6	✓ 30.143 ICMP	✓ 30.561 ICMP
reply 3	before the pings	✓ 30.831 ICMP	✓ 31.424 SRv6	✓ 30.389 SRv6	✓ 30.645 ICMP	✓ 31.063 ICMP
request 3	before the pings	✓ 30.829 ICMP	✓ 31.423 SRv6	✓ 30.388 SRv6	✓ 30.644 ICMP	✓ 31.062 ICMP
reply 4	before the pings	✓ 31.332 ICMP	✓ 31.926 SRv6	✓ 30.891 SRv6	✓ 31.146 ICMP	✓ 31.564 ICMP

1. dc1-h1 host
sends to 172.20.3.0/24
172.20.1.2 on 172.20.3.0/24

6. ce3 ce eth1

7. dc3-h1 host

A capture along a path: every ping at every hop

13. Stop 3 — Nautobot, the source of truth

Nautobot holds the lab as a model: devices, interfaces, cables, IP addresses, VRFs with their RDs and RTs, BGP peerings. It is seeded from `lab.conf`, and the lab checks itself against it both ways — the configuration Nautobot would render must equal the one `lab.conf` renders, and every line must be present on the routers.

>>> nautobot

<

FAVORITES

DEVICES

IPAM

ORGANIZATION

CIRCUITS

VPN

ROUTING

POWER

WIRELESS

SECRETS

LOAD BALANCERS

VIRTUALIZATION

CLOUD

APPROVALS

EXTENSIBILITY

GOLDEN CONFIG

JOBS

APPS

cat9000v lab NMS

>>> pe1

in: Devices

Press Ctrl+K to search

?

admin

Devices / srv6-core / dc1

Device

Advanced

Contacts 0

Dynamic Groups 1

Object Metadata 0

Data Compliance

Interfaces 7

Status

Sept. 16, 2026 4:25 p.m.

26 minutes ago

+ Add Components

Edit Device

DEVICE

Location

srv6-core (Site)

dc1 (Data Center)

Rack

Position (U)

Tenant

Device Family

Device Type

VyOS VyOS (1U)

Serial Number

Asset Tag

MANAGEMENT

Role

srv6-pe

Platform

vyos

Status

Active

Primary IPv4

10.3.0.11/24 (Global)

Primary IPv6

ASSIGNED VRFS 2

VRF

Namespace

VRF RD

Tenant

tenant-a

Global

65000:101

tenant-a*

tenant-b

Global

65000:201

tenant-b*

CLUSTERS 0

Name

Cluster Type

Cluster Group

Tenant

VMs

— No clusters found —

+ Add cluster

SERVICES 0

Name

Protocol

Ports

IP addresses

Description

— No services found —

+ Add service

IMAGES 0

Name

Size

Created

— No image attachments found —

+ Add image attachment

pe1 in Nautobot

>>> nautobot

<

FAVORITES

DEVICES

IPAM

ORGANIZATION

CIRCUITS

VPN

ROUTING

POWER

WIRELESS

SECRETS

LOAD BALANCERS

VIRTUALIZATION

CLOUD

APPROVALS

EXTENSIBILITY

GOLDEN CONFIG

JOBS

APPS

cat9000v lab NMS

>>> tenant-a

in: VRFS

Press Ctrl+K to search

?

admin

VRFS

VRF

Advanced

Contacts 0

Object Metadata 0

Data Compliance

Notes 0

Change Log

Sept. 16, 2026 4:24 p.m.

20 minutes ago

Edit VRF

VRF

Name

tenant-a

Route Distinguisher

Status

Active

Namespace

Global

Tenant

srv6-core / tenant-a

Description

tenant-a: L3VPN over SRv6 (End.DT4), kernel table 100 on every PE and CE

TAGS

No tags assigned

IMPORT ROUTE TARGETS 1

Name

Tenant

Description

65000:100

tenant-a

tenant-a import/export

EXPORT ROUTE TARGETS 1

Name

Tenant

Description

65000:100

tenant-a

tenant-a import/export

ASSIGNED PREFIXES 8

Prefix

Type

Status

VRFS

Tenant

Locations

VLAN

Role

Description

172.16.1.0/30

Network

Active

tenant-a

tenant-a

dc1

—

attachment-circuit

tenant-a attachment circuit pe1 eth3 <-> ce1 eth1

172.16.2.0/30

Network

Active

tenant-a

tenant-a

dc2

—

attachment-circuit

tenant-a attachment circuit pe2 eth3 <-> ce2 eth1

172.16.3.0/30

Network

Active

tenant-a

tenant-a

dc3

—

attachment-circuit

tenant-a attachment circuit pe3 eth3 <-> ce3 eth1

172.16.4.0/30

Network

Active

tenant-a

tenant-a

dc4

—

attachment-circuit

tenant-a attachment circuit pe4 eth3 <-> ce4 eth1

The tenant VRFS, with route distinguishers and route targets

14. Stop 4 — monitoring and alerting

The NMS VM runs **Prometheus** (scraping node-exporter and frf-exporter on every router, node-exporter on every host, and the portal's own `/metrics`), **VictoriaMetrics** for longer retention, **VictoriaLogs** for every router's syslog, and **Grafana** for dashboards. Alert rules cover the things an operator cares about: a host or a CE session down, an IS-IS or BFD adjacency missing, a VPN session down, a tenant outside its SLA, failed tests, and a notice when there is no room left for another tenant. Portal runs, steering changes and SLA state changes all appear as annotations on every dashboard.

15. Stop 5 — tests and CI

`./lab.sh test` runs sixteen Robot Framework suites — 109 checks — against the live lab: management, the underlay, SRv6, the VPNs, end to end reachability, route-reflector redundancy, steering, failover (a link cut with BFD: 4 packets lost), Nautobot, throughput, monitoring, dual-stack, internet breakout, the looking glass and the portal's operations features. Before and after, every router's configuration and routing tables are saved; the difference must be empty. Each push to GitHub is mirrored to a local Gitea whose runner validates the configurations and runs the suites.

Walk a packet through it

16. Before you start

Open two terminals on the lab host. Everything below uses `./lab.sh ssh <node>` to log in (VyOS routers: `vyos / vyos`; hosts: `lab / lab`). On a VyOS router, `pe1>` marks the operational CLI (`show ...`) and `pe1$` the Linux shell underneath (`ip ...`, `tcpdump`), where the SRv6 forwarding actually happens, where the SRv6 forwarding actually happens. We follow one ping from **dc1-h1** (tenant-a, data centre 1) to **dc3-h1** (tenant-a, data centre 3).

17. Step 1 — the customer's view

On dc1-h1 the world is small: an address on its LAN and a default route to its CE.

```
dc1-h1$ ip -br addr; ip route
lo                UNKNOWN      127.0.0.1/8 ::1/128
eth0              UP          10.3.0.41/24 fe80::5054:ff:fec6:c00/64
eth1              UP          172.20.1.2/24 fd00:20:1::2/64 fe80::5054:ff:fec6:c01/64
default via 172.20.1.1 dev eth1
10.0.0.0/8 via 10.3.0.1 dev eth0
10.3.0.0/24 dev eth0 proto kernel scope link src 10.3.0.41
172.20.1.0/24 dev eth1 proto kernel scope link src 172.20.1.2
```

It reaches the tenant-a host in data centre 3...

```
dc1-h1$ ping -c 3 172.20.3.2
PING 172.20.3.2 (172.20.3.2): 56 data bytes
64 bytes from 172.20.3.2: seq=0 ttl=42 time=1.970 ms
64 bytes from 172.20.3.2: seq=1 ttl=42 time=1.966 ms
64 bytes from 172.20.3.2: seq=2 ttl=42 time=2.039 ms

--- 172.20.3.2 ping statistics ---
3 packets transmitted, 3 packets received, 0% packet loss
round-trip min/avg/max = 1.966/1.991/2.039 ms
```

...and **cannot** reach the tenant-b host in its own data centre, one hop away on the same CE:

```
dc1-h1$ ping -c 2 172.21.1.2
PING 172.21.1.2 (172.21.1.2): 56 data bytes

--- 172.21.1.2 ping statistics ---
2 packets transmitted, 0 packets received, 100% packet loss
```

The traceroute shows the customer never sees the core: the hop between the two PEs is invisible (*), because inside the core the packet is inside an IPv6 envelope.

```
dc1-h1$ traceroute -n 172.20.3.2
traceroute to 172.20.3.2 (172.20.3.2), 30 hops max, 46 byte packets
 1  172.20.1.1  0.004 ms
 2  *
 3  172.16.3.2  1.728 ms
 4  172.20.3.2  1.709 ms
```

18. Step 2 — the underlay: IS-IS and the locators

On pe1, IS-IS has two neighbours — the P routers it is cabled to:

```
pe1> show isis neighbor
Area VyOS:
System Id      Interface  L  State      Holdtime  SNPA
p1             eth1      2  Up         29        2020.2020.2020
p2             eth2      2  Up         28        2020.2020.2020
```

pe1 owns one locator, and it is up:

```
pe1> show segment-routing srv6 locator
Locator:
Name           ID      Prefix          Status
-----
main           1 fd00:c1::/48    Up
```

Through IS-IS, pe1 has learnt every other router's locator as an ordinary IPv6 route. pe3's `fd00:c3::/48` goes via p2 (the `fe80::...603` neighbour on eth2) — the IGP shortest path:

```
pe1$ vtysh -c 'show ipv6 route isis' | grep fd00:c | head -12
I>* fd00:c1::/48 [115/0] is directly connected, dum0, seg6local uN, weight 1, 00:37:38
I>* fd00:c1:e002::/64 [115/0] is directly connected, eth2, seg6local uA nh6 fe80::5054:ff:fec6:603, eth2, weight 1, 00:37:38
I>* fd00:c1:e003::/64 [115/0] is directly connected, eth1, seg6local uA nh6 fe80::5054:ff:fec6:503, eth1, weight 1, 00:37:38
I>* fd00:c2::/48 [115/20] via fe80::5054:ff:fec6:503, eth1, weight 1, 00:37:11
I>* fd00:c2::1/128 [115/30] via fe80::5054:ff:fec6:503, eth1, weight 1, 00:37:11
I>* fd00:c3::/48 [115/20] via fe80::5054:ff:fec6:603, eth2, weight 1, 00:37:12
I>* fd00:c3::1/128 [115/30] via fe80::5054:ff:fec6:603, eth2, weight 1, 00:37:12
I>* fd00:c4::/48 [115/20] via fe80::5054:ff:fec6:603, eth2, weight 1, 00:37:12
I>* fd00:c4::1/128 [115/30] via fe80::5054:ff:fec6:603, eth2, weight 1, 00:37:12
I>* fd00:c11::/48 [115/10] via fe80::5054:ff:fec6:503, eth1, weight 1, 00:37:37
I>* fd00:c11::1/128 [115/20] via fe80::5054:ff:fec6:503, eth1, weight 1, 00:37:37
I>* fd00:c12::/48 [115/10] via fe80::5054:ff:fec6:603, eth2, weight 1, 00:37:37
```

19. Step 3 — the instructions pe1 will carry out

The kernel's table of local SIDs is where behaviours become real. On pe1: the node SID (End, *uN*) on the whole locator, one End.X (*uA*) per core link, and one End.DT46 per tenant VRF:

```
pe1$ ip -6 route show | grep seg6local
fd00:c1:e000:: nhid 45  encap seg6local action End.DT46 vrftable tenant-b dev tenant-b proto bgp metric 20 pref medium
fd00:c1:e001:: nhid 39  encap seg6local action End.DT46 vrftable tenant-a dev tenant-a proto bgp metric 20 pref medium
fd00:c1:e002::/64 nhid 42  encap seg6local action End.X nh6 fe80::5054:ff:fec6:603 oif eth2 flavors next-csid lb len 32 nflen 16 dev eth2 proto isis metric 20 pref medium
fd00:c1:e003::/64 nhid 43  encap seg6local action End.X nh6 fe80::5054:ff:fec6:503 oif eth1 flavors next-csid lb len 32 nflen 16 dev eth1 proto isis metric 20 pref medium
fd00:c1::/48 nhid 37  encap seg6local action End flavors next-csid lflen 32 nflen 16 dev dum0 proto isis metric 20 pref medium
```

Read it as a sentence. "fd00:c1:e00x:: ... action End.DT46 vrftable tenant-a" means: a packet addressed to this SID is to be unwrapped and looked up in tenant-a's table. The function values (e000, e001 ...) are chosen by FRR when the SIDs are allocated, so they can differ between your lab and this guide.

20. Step 4 — BGP: which SID leads to which customer

pe3 learnt `172.20.3.0/24` from ce3, allocated an End.DT46 SID for tenant-a, and advertised both to the route reflectors. pe1 receives the route twice (once from each reflector), tagged with route target `65000:100` — which is why only VRF tenant-a imports it:

```
pe1> show bgp ipv4 vpn 172.20.3.0/24
BGP routing table entry for 65000:103:172.20.3.0/24, version 2
not allocated
Paths: (2 available, best #1)
  Not advertised to any peer
  65003
    0.0.0.0 (metric 20) from fd00:a::11 (10.255.0.3)
      Origin IGP, metric 0, localpref 100, valid, internal, multipath, best (Neighbor IP)
      Extended Community: RT:65000:100
      Originator: 10.255.0.3, Cluster list: 10.255.0.11
      Remote labels: 917520
      Remote SID: fd00:c:3::, sid structure=[32 16 16 0 16 48]
      Last update: Tue Sep 29 11:37:16 2026
  65003
    0.0.0.0 (metric 20) from fd00:a::13 (10.255.0.3)
      Origin IGP, metric 0, localpref 100, valid, internal, multipath
      Extended Community: RT:65000:100
      Originator: 10.255.0.3, Cluster list: 10.255.0.13
      Remote labels: 917520
      Remote SID: fd00:c:3::, sid structure=[32 16 16 0 16 48]
      Last update: Tue Sep 29 11:37:16 2026
```

The "Remote SID" line and the "Remote labels" value together make the full SID: BGP sends the locator part in the SID field and the function in the label field ("transposition"), which saves space when many routes share a locator.

21. Step 5 — the VRF: where encapsulation is decided

pe1's tenant-a table now says, for `172.20.3.0/24`: *wrap it in SRv6, one segment, pe3's tenant-a SID, out towards p2*. Local sites go straight to the CE; remote ones all have an `encap seg6` instruction:

```

pe1$ ip route show vrf tenant-a
default nhid 105 encap seg6 mode encap segs 1 [ fd00:c:4:e001:: ] via inet6 fe80::5054:ff:fec6:603 dev eth2 proto bgp metric 20
127.0.0.0/8 dev tenant-a proto kernel scope link src 127.0.0.1
172.16.1.0/30 dev eth3 proto kernel scope link src 172.16.1.1
172.16.2.0/30 nhid 97 proto bgp metric 20
    nexthop encap seg6 mode encap segs 1 [ fd00:c:2:e001:: ] via inet6 fe80::5054:ff:fec6:503 dev eth1 weight 1
172.16.3.0/30 nhid 102 encap seg6 mode encap segs 1 [ fd00:c:3:e001:: ] via inet6 fe80::5054:ff:fec6:603 dev eth2 proto bgp metric 20
172.16.4.0/30 nhid 105 encap seg6 mode encap segs 1 [ fd00:c:4:e001:: ] via inet6 fe80::5054:ff:fec6:603 dev eth2 proto bgp metric 20
172.16.5.0/30 nhid 105 encap seg6 mode encap segs 1 [ fd00:c:4:e001:: ] via inet6 fe80::5054:ff:fec6:603 dev eth2 proto bgp metric 20
172.20.1.0/24 nhid 79 via 172.16.1.2 dev eth3 proto bgp metric 20
172.20.2.0/24 nhid 97 proto bgp metric 20
    nexthop encap seg6 mode encap segs 1 [ fd00:c:2:e001:: ] via inet6 fe80::5054:ff:fec6:503 dev eth1 weight 1
    nexthop encap seg6 mode encap segs 1 [ fd00:c:2:e001:: ] via inet6 fe80::5054:ff:fec6:603 dev eth2 weight 1
172.20.3.0/24 nhid 102 encap seg6 mode encap segs 1 [ fd00:c:3:e001:: ] via inet6 fe80::5054:ff:fec6:603 dev eth2 proto bgp metric 20
172.20.4.0/24 nhid 105 encap seg6 mode encap segs 1 [ fd00:c:4:e001:: ] via inet6 fe80::5054:ff:fec6:603 dev eth2 proto bgp metric 20
172.24.1.1 nhid 79 via 172.16.1.2 dev eth3 proto bgp metric 20
172.24.1.2 nhid 79 via 172.16.1.2 dev eth3 proto bgp metric 20
172.24.1.3 nhid 79 via 172.16.1.2 dev eth3 proto bgp metric 20
172.24.1.4 nhid 79 via 172.16.1.2 dev eth3 proto bgp metric 20
172.24.1.5 nhid 79 via 172.16.1.2 dev eth3 proto bgp metric 20
172.24.1.6 nhid 79 via 172.16.1.2 dev eth3 proto bgp metric 20
172.24.1.7 nhid 79 via 172.16.1.2 dev eth3 proto bgp metric 20
... (53 more lines)

```

22. Step 6 — on the wire in the core

Capture on p2 while dc1-h1 pings. Each packet is an IPv6 packet from pe1's loopback to pe3's SID, with a one-entry SRH (RT6 ... type=4), and the customer's IPv4 packet inside it:

```

p2$ sudo tcpdump -ni eth5 -c 4 -vv 'ip6 and dst net fd00:c:3::/48' # eth5 = the link to pe3
12:15:03.387621 IP6 (hlim 62, next-header Routing (43) payload length: 108) fd00:a::1 > fd00:c:3:e001:: RT6 (len=2, type=4, segleft=0, last-entry=0, flags=0x0, tag=0, [0]fd00:c:3:e001::) IP (tos 0x0, ttl 63, id 40595, offset 0, flags [DF], proto ICMP (1), length 84)
    172.20.1.2 > 172.20.3.2: ICMP echo request, id 103, seq 0, length 64
12:15:03.887745 IP6 (hlim 62, next-header Routing (43) payload length: 108) fd00:a::1 > fd00:c:3:e001:: RT6 (len=2, type=4, segleft=0, last-entry=0, flags=0x0, tag=0, [0]fd00:c:3:e001::) IP (tos 0x0, ttl 63, id 40650, offset 0, flags [DF], proto ICMP (1), length 84)
    172.20.1.2 > 172.20.3.2: ICMP echo request, id 103, seq 1, length 64
12:15:04.387867 IP6 (hlim 62, next-header Routing (43) payload length: 108) fd00:a::1 > fd00:c:3:e001:: RT6 (len=2, type=4, segleft=0, last-entry=0, flags=0x0, tag=0, [0]fd00:c:3:e001::) IP (tos 0x0, ttl 63, id 40929, offset 0, flags [DF], proto ICMP (1), length 84)
    172.20.1.2 > 172.20.3.2: ICMP echo request, id 103, seq 2, length 64
12:15:04.887947 IP6 (hlim 62, next-header Routing (43) payload length: 108) fd00:a::1 > fd00:c:3:e001:: RT6 (len=2, type=4, segleft=0, last-entry=0, flags=0x0, tag=0, [0]fd00:c:3:e001::) IP (tos 0x0, ttl 63, id 41188, offset 0, flags [DF], proto ICMP (1), length 84)
    172.20.1.2 > 172.20.3.2: ICMP echo request, id 103, seq 3, length 64

```

And p2 has no idea who the customers are. It has no VRFs, and its IPv4 table holds only the management network:


```
p2$ ip route show vrf tenant-a; ip vrf show
Error: argument "tenant-a" is wrong: Invalid VRF

Name          Table
-----
No VRF has been configured
p2$ ip route show          # IPv4
10.0.0.0/8 nhid 23 via 10.3.0.1 dev eth0 proto static metric 20
10.3.0.0/24 dev eth0 proto kernel scope link src 10.3.0.22
```

That is the whole point of SRv6: the core carries VPN traffic with nothing but IPv6 routing.

23. Step 7 — delivery at pe3

pe3's SID table has the matching End.DT46 entries. A packet addressed to the tenant-a one is unwrapped and handed to VRF tenant-a, which knows `172.20.3.0/24` is behind ce3:

```
pe3$ ip -6 route show | grep End.DT4
fd00:c3:e000:: nhid 45  encap seg6local action End.DT46 vrftable tenant-b dev tenant-b proto bgp metric 20 pref
medium
fd00:c3:e001:: nhid 39  encap seg6local action End.DT46 vrftable tenant-a dev tenant-a proto bgp metric 20 pref
medium
```

24. Step 8 — choose the path yourself

By default everything west↔east crosses p2. Pin tenant-b's traffic from pe1 to `172.21.3.0/24` onto the long way round, p1 → p3 → pe3. The steering tool builds a single uSID carrier from the path:

```
$ ./lab.sh steer add pe1 tenant-b 172.21.3.0/24 p1 p3
pe1: tenant-b 172.21.3.0/24 -> p1 -> p3 -> pe3  segments fd00:c11:13:3:e000:: (out eth1) [uSID: one compressed
segment]
172.21.3.0/24 nhid 142  encap seg6 mode encap segs 1 [ fd00:c11:13:3:e000:: ] dev eth1 proto static metric 20
```

Now capture on p1 and then on p3 while dc1-h2 pings dc3-h2. Watch the destination address shrink by one micro-SID at each hop — exactly Figure 5:

```
p1$ sudo tcpdump -ni eth2 -c 2 -vv 'ip6 and dst net fd00:c::/32' # leaving p1 towards p3: its own l1 consumed
12:16:19.500122 IP6 (hlim 62, next-header Routing (43) payload length: 108) fd00:a::1 > fd00:c13:3:e000::: RT6
(len=2, type=4, segleft=0, last-entry=0, flags=0x0, tag=0, [0]fd00:c11:13:3:e000::) IP (tos 0x0, ttl 63, id 6396
8, offset 0, flags [DF], proto ICMP (1), length 84)
172.21.1.2 > 172.21.3.2: ICMP echo request, id 100, seq 0, length 64
12:16:20.001520 IP6 (hlim 62, next-header Routing (43) payload length: 108) fd00:a::1 > fd00:c13:3:e000::: RT6
(len=2, type=4, segleft=0, last-entry=0, flags=0x0, tag=0, [0]fd00:c11:13:3:e000::) IP (tos 0x0, ttl 63, id 6408
8, offset 0, flags [DF], proto ICMP (1), length 84)
172.21.1.2 > 172.21.3.2: ICMP echo request, id 100, seq 1, length 64
```

```
p3$ sudo tcpdump -ni eth3 -c 2 -vv 'ip6 and dst net fd00:c::/32' # leaving p3 towards pe3: only pe3's SID is le
ft
12:16:23.624088 IP6 (hlim 63, next-header Routing (43) payload length: 108) fd00:a::3 > fd00:c4:e000::: RT6 (len
=2, type=4, segleft=0, last-entry=0, flags=0x0, tag=0, [0]fd00:c4:e000::) IP (tos 0x0, ttl 63, id 52453, offset
0, flags [none], proto ICMP (1), length 84)
172.21.3.2 > 172.21.4.2: ICMP echo reply, id 101, seq 0, length 64
12:16:23.624885 IP6 (hlim 61, next-header Routing (43) payload length: 108) fd00:a::1 > fd00:c3:e000::: RT6 (len
=2, type=4, segleft=0, last-entry=0, flags=0x0, tag=0, [0]fd00:c11:13:3:e000::) IP (tos 0x0, ttl 63, id 64832, o
ffset 0, flags [DF], proto ICMP (1), length 84)
172.21.1.2 > 172.21.3.2: ICMP echo request, id 102, seq 0, length 64
```

The reply still takes the shortest path back through p2: steering is per direction. Remove the policy and traffic returns to the IGP path:

```
$ ./lab.sh steer del pe1 tenant-b 172.21.3.0/24
pe1: tenant-b 172.21.3.0/24 back on the IGP shortest path
172.21.3.0/24 nhid 113 encap seg6 mode encap segs 1 [ fd00:c3:e000:: ] via inet6 fe80::5054:ff:fec6:603 dev eth
2 proto bgp metric 20
```

Same thing, in the portal. In the portal's Steering view, choose pe1, tenant-b, 172.21.3.0/24, via p1 p3 , and press **Preview** to see the path on the map; **Apply** installs it; **Measure delay** compares it with the IGP path (the long way is roughly one millisecond longer).

25. Step 9 — break something

Every core adjacency runs **BFD**, which notices a dead neighbour in under a second (the links are tunnels that never lose carrier, so without BFD it would take IS-IS's 30-second hold time):

```
pe1> show bfd peers brief
Session count: 2
SessionId  LocalAddress                PeerAddress                Status    Profi
le
=====  =====
==
2026629426 fe80::5054:ff:fec6:101        fe80::5054:ff:fec6:503    up        -
3077310471 fe80::5054:ff:fec6:102        fe80::5054:ff:fec6:603    up        -
```

Suite 08 (`./lab.sh test suites/08_failover.robot`) cuts the p2–pe3 link silently while a ping runs every 0.2 s: pe3's routes move to p3 within a second, about four packets are lost, and everything comes back when the link is restored. Try it, and watch the looking glass's History view and the SLA matrix while it happens.

26. Where to go next

- **Exercises.** `docs/session/exercises.md` has break-and-fix exercises for the lab.
- **The deep dive.** `docs/srv6-walkthrough.pdf` goes table by table through dual-stack (End.DT46 for IPv6), the internet breakout, the Linux quirk the PEs work around, and more.
- **Add a tenant** from the portal and follow the run — then find its new SIDs on the PEs, its routes in the looking glass, and its sites in the SLA matrix.
- **Standards.** RFC 8402 (Segment Routing architecture), RFC 8754 (the SRH), RFC 8986 (SRv6 network programming — the behaviours), RFC 9252 (BGP services over SRv6 — the L3VPN), RFC 9352 (IS-IS extensions for SRv6), RFC 9800 (compressed SIDs — uSID).